

Databases and SQL

SQL 5 -- Aggregate Queries

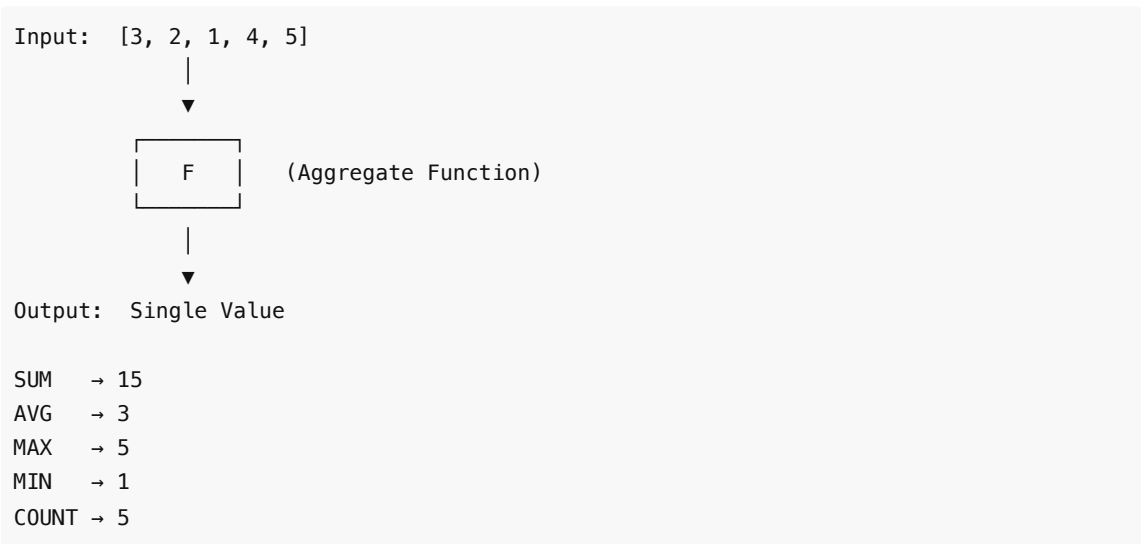
Presented By: Rehber Moin

Agenda

- Aggregate Functions (COUNT, AVG, MIN, MAX, SUM)
 - GROUP BY
 - HAVING
 - Cross Join (revision)
 - Implicit Join
 - JOIN with WHERE vs ON
-

Precap: What Are Aggregate Functions?

Aggregate functions take **multiple data points** as input and return a **single data point** as output.



Aggregate Functions

The Five Core Aggregates

Function	Purpose	Example
SUM(col)	Sum of all values in the column	SELECT SUM(psp) FROM students;
COUNT(*)	Number of rows	SELECT COUNT(*) FROM students;
MIN(col)	Smallest value in the column	SELECT MIN(psp) FROM students;
MAX(col)	Largest value in the column	SELECT MAX(psp) FROM students;
AVG(col)	Average of all values in the column	SELECT AVG(psp) FROM students;

Example

```
USE scaler;

SELECT SUM(psp) FROM students;    -- e.g., 613
SELECT COUNT(*) FROM students;    -- e.g., 8
SELECT MIN(psp) FROM students;    -- e.g., 80
SELECT MAX(psp) FROM students;    -- e.g., 92
SELECT AVG(psp) FROM students;    -- e.g., 86.14
```

Rule 1: Aggregate Functions Ignore NULL Values

```
-- If one student has name = NULL:
SELECT COUNT(name) FROM students;    -- Returns 7 (not 8)
SELECT COUNT(*) FROM students;       -- Returns 8 (counts rows, not column values)
```

`COUNT(*)` counts **rows**. `COUNT(column)` counts **non-NULL values** in that column.

The COUNT(10) Puzzle

```
SELECT COUNT(10) FROM students;    -- Returns 8. Why?
```

To understand, run:

```
SELECT 10 FROM students;
-- Returns one row of "10" for each student (8 rows of the literal value 10).
-- COUNT(10) counts those 8 non-NULL values → 8.
```

`COUNT(expression)` counts how many times the expression evaluates to a non-NULL value. Since `10` is never NULL, it counts every row.

Rule 2: Cannot Mix Aggregated and Non-Aggregated Columns

```
-- This does NOT work:
SELECT name, SUM(psp) FROM students;
```

Why? `SUM(psp)` produces **1 data point** (a single number). `name` produces **8 data points** (one per row). There's no way to map a single sum to 8 different names.

name (8 rows)	SUM(psp) (1 value)	
Schumacher		
Nikki		
Verstappen	613	← How to map 1 value to 8 rows?
Hulkenberg		
Charles		

...	
-----	--

Valid: You can combine multiple aggregates:

```
SELECT AVG(batch_id), SUM(bsp) FROM students; -- Both produce 1 value. OK.
```

DISTINCT Inside Aggregates

```
-- Count the number of distinct batches that are running
SELECT COUNT(DISTINCT batch_id) AS 'Batch Count' FROM students;
```

GROUP BY

GROUP BY splits rows into **groups** based on one or more columns, then applies aggregate functions **per group**.

The Problem

Can we count the number of students **in each batch**?

Without **GROUP BY**, **COUNT(*)** gives us the total count across the entire table. We need it **per batch**.

Syntax

```
SELECT batch_id, COUNT(*)
FROM students
GROUP BY batch_id;
```

How It Works

Original Table	After GROUP BY batch_id																												
<table><tr><th>id</th><th>name</th><th>psp</th><th>b_id</th></tr><tr><td>1</td><td>Schumacher</td><td>87</td><td>1</td></tr><tr><td>2</td><td>Nikki</td><td>90</td><td>1</td></tr><tr><td>3</td><td>Verstappen</td><td>92</td><td>2</td></tr><tr><td>4</td><td>Hulkenberg</td><td>87</td><td>1</td></tr><tr><td>5</td><td>Charles</td><td>87</td><td>3</td></tr><tr><td>...</td><td></td><td></td><td></td></tr></table>	id	name	psp	b_id	1	Schumacher	87	1	2	Nikki	90	1	3	Verstappen	92	2	4	Hulkenberg	87	1	5	Charles	87	3	...				Group b_id=1: [Schumacher, Nikki, Hulkenberg] → Group b_id=2: [Verstappen] → Group b_id=3: [Charles] → Group b_id=NULL: [...] →
id	name	psp	b_id																										
1	Schumacher	87	1																										
2	Nikki	90	1																										
3	Verstappen	92	2																										
4	Hulkenberg	87	1																										
5	Charles	87	3																										
...																													
COUNT=3																													
COUNT=1																													
COUNT=1																													
COUNT=...																													
	Result:																												
	<table><tr><th>batch_id</th><th>COUNT(*)</th></tr><tr><td>1</td><td>3</td></tr><tr><td>2</td><td>1</td></tr></table>	batch_id	COUNT(*)	1	3	2	1																						
batch_id	COUNT(*)																												
1	3																												
2	1																												

3	1
NULL	...

Multiple Aggregates per Group

```
-- Count and max PSP per batch
SELECT batch_id, COUNT(*), MAX(psp)
FROM students
GROUP BY batch_id;
```

Rule: SELECT Can Only Contain GROUP BY Columns or Aggregates

```
-- This does NOT work:
SELECT name, batch_id FROM students
GROUP BY batch_id;
```

Why? Batch 1 has 3 students (Schumacher, Nikki, Hulkenberg). Which `name` should SQL pick for the group? It's ambiguous.

Group batch_id=1:

Schumacher
Nikki
Hulkenberg

← Which name to show? Ambiguous!

Fix: You can only SELECT columns that are in the GROUP BY clause or inside aggregate functions.

```
-- This WORKS: both columns are in GROUP BY
SELECT name, batch_id FROM students
GROUP BY batch_id, name;
-- Groups are now (batch_id, name) pairs – each is unique.
```

More Examples

```
-- Average PSP per batch
SELECT batch_id, AVG(psp) FROM students
GROUP BY batch_id;

-- Number of movies released per year (Sakila)
USE sakila;
SELECT release_year, COUNT(*)
FROM film
GROUP BY release_year;
```

HAVING -- Filtering Groups

The Problem

Find all batches that have more than 2 students.

Your first instinct might be:

```
-- This does NOT work:
SELECT batch_id, COUNT(*)
FROM students
GROUP BY batch_id
WHERE COUNT(*) > 2;      -- ERROR!
```

Why? `WHERE` filters **rows** before grouping. At that point, groups don't exist yet, so `COUNT(*)` is meaningless.

The Solution: HAVING

`HAVING` filters **groups** after they are formed (just like `WHERE` filters rows before grouping).

```
SELECT batch_id, COUNT(*)
FROM students
GROUP BY batch_id
HAVING COUNT(*) >= 2;
```

WHERE vs HAVING

SQL Execution Order

FROM	→	Which table(s)?	
WHERE	→	Filter individual ROWS	(before grouping)
GROUP BY	→	Form groups	
HAVING	→	Filter GROUPS	(after grouping)
SELECT	→	Choose columns / aggregates	
ORDER BY	→	Sort the result	
LIMIT	→	Restrict output count	

Clause	Filters	When	Can Use Aggregates?
WHERE	Rows	Before GROUP BY	No
HAVING	Groups	After GROUP BY	Yes

Combining WHERE and HAVING

```
-- Only include students whose name starts with 'H',
-- then find batches with at least 1 such student.
```

```
SELECT batch_id, COUNT(*)
FROM students
WHERE name LIKE 'H%'           -- Filter ROWS first (before grouping)
GROUP BY batch_id
HAVING COUNT(*) >= 1;         -- Filter GROUPS after
```

Order of execution:

1. WHERE name LIKE 'H%' -- keep only students with names starting with 'H'
2. GROUP BY batch_id -- group the remaining students by batch
3. HAVING COUNT(*) >= 1 -- keep only groups with at least 1 student

GROUP BY + JOIN

You can combine GROUP BY with JOINs to get richer results.

Example: Average PSP by Batch (with Batch Name)

```
USE scaler;

SELECT b.id, b.name, AVG(s.psp), COUNT(*)
FROM students s
JOIN batches b ON b.id = s.batch_id
GROUP BY b.id, b.name;
```

Result:

id	name	AVG(psp)	COUNT(*)
1	LLD	88.00	3
2	SQL	92.00	1
3	HLD	87.00	1

With ORDER BY

```
-- Sorted by average PSP ascending
SELECT b.id, b.name, AVG(s.psp), COUNT(*)
FROM students s
JOIN batches b ON b.id = s.batch_id
GROUP BY b.id, b.name
ORDER BY AVG(s.psp) ASC;
```

Sakila Example: Rental Durations with More Than 200 Movies

```
USE sakila;
```

```

-- Step 1: See how many films per rental duration
SELECT rental_duration, COUNT(*)
FROM film
GROUP BY rental_duration;

-- Step 2: Filter to only rental durations with > 200 films
SELECT rental_duration, COUNT(*)
FROM film
GROUP BY rental_duration
HAVING COUNT(*) > 200;

```

Sakila Example: Customers with 30+ Rentals

Following the approach framework:

```

-- Step 1: Identify tables → rental, customer
-- Step 2: JOIN them
-- Step 3: GROUP BY customer
-- Step 4: HAVING for the filter
-- Step 5: ORDER BY and SELECT columns

SELECT
    rent.customer_id AS 'Customer ID',
    CONCAT(cust.first_name, ' ', cust.last_name) AS 'Full Name',
    COUNT(*) AS 'Movies Rented'
FROM rental rent
JOIN customer cust ON rent.customer_id = cust.customer_id
GROUP BY rent.customer_id
HAVING COUNT(*) > 30
ORDER BY COUNT(*) DESC;

```

Cross Join (Revision)

A Cross Join pairs every row from Table 1 with every row from Table 2. No `ON` condition needed.

Use Case: Generate All Combinations

Example: You have T-shirt designs and sizes and want every possible combination.

Designs: [Logo, Stripes, Plain]
 Sizes: [S, M, L, XL]

Cross Join → 3 x 4 = 12 combinations:
 (Logo, S), (Logo, M), (Logo, L), (Logo, XL),
 (Stripes, S), (Stripes, M), ...

```

SELECT * FROM students s JOIN batches b;
-- OR equivalently:
SELECT * FROM students s CROSS JOIN batches b;

```

Implicit Join

An **Implicit Join** is syntactic sugar -- a shorthand way to write a Cross Join using commas instead of the `JOIN` keyword.

```
-- Explicit Cross Join:
SELECT * FROM students s JOIN batches b;

-- Implicit Join (same result):
SELECT * FROM students s, batches b;
```

Both produce a Cross Join (Cartesian product). To make it an Inner Join, add a `WHERE` clause:

```
-- Implicit Inner Join:
SELECT * FROM students s, batches b
WHERE b.id = s.batch_id;
```

Note: The implicit join syntax is older SQL style. The explicit `JOIN ... ON` syntax is preferred because it clearly separates the **join condition** from the **filter condition**.

JOIN with WHERE vs ON (Revisited)

Using ON (Standard)

```
SELECT * FROM students
JOIN batches
ON batches.id = students.batch_id;
```

The `ON` clause defines the **join condition** -- how the two tables relate.

Using WHERE (Older Style)

```
SELECT * FROM students
JOIN batches
WHERE batches.id = students.batch_id;
```

For **INNER JOIN**, these produce the same result.

Key Difference (Outer Joins)

For **Outer Joins**, `ON` and `WHERE` behave differently (covered in SQL 4):

```
ON clause:  Determines whether rows MATCH during the join.
            Unmatched outer-table rows still appear (with NULLs).

WHERE clause: Filters AFTER the join.
            Removes rows entirely (can undo the outer join effect).
```

Recommendation

Situation	Use	Why
Join logic	ON	Clearly states how tables relate
Row filters	WHERE	Clearly states which rows to keep
Outer Joins	ON only for join condition	WHERE would turn it into an Inner Join

Summary

Topic	Key Takeaway
Aggregate Functions	Take multiple values, return one. Ignore NULLs.
COUNT(*) vs COUNT(col)	COUNT(*) counts rows; COUNT(col) counts non-NULL values.
Aggregated + Non-Aggregated	Cannot mix them in SELECT without GROUP BY.
GROUP BY	Splits rows into groups. SELECT can only contain GROUP BY columns or aggregates.
HAVING	Filters groups (after GROUP BY). WHERE filters rows (before GROUP BY).
Execution Order	FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT
Cross Join	Every row x every row. Useful for generating all combinations.
Implicit Join	Comma-separated tables = Cross Join. Add WHERE for Inner Join.
ON vs WHERE	Same for Inner Join. Different for Outer Join (ON for join logic, WHERE for filtering).